🛠️ Implementation Blueprint

Module 1: Data Ingestion & Graph Construction

- **Tools:** PyVerilog, Yosys, NetworkX, PyTorch Geometric (PyG)
- **Steps:**
    1. Parse RTL/netlist → Abstract Syntax Tree (AST).
    2. Generate **multi-view graphs**:
        - Structural graph (gates + connections).
        - Functional graph (signal/control flow).
        - Temporal graph (toggle rates from simulation traces).
    3. Store graphs in PyG Data objects:

```
python
data = Data(
  x=node_features,      # [num_nodes, num_features]
  edge_index=edge_list,  # [2, num_edges]
  y=graph_label        # 0 or 1
)
```

Module 2: Feature Engineering

- **Features per node:**
    - Gate type (one-hot encoding).
    - Degree (in/out).
    - Activation probability (from simulation).
    - Distance to outputs (shortest path).
    - Toggle rate, entropy, fan-in/fan-out.
- **Output:** Node feature matrix [num_nodes, num_features].

Module 3: Multi-View GNN Ensemble

- **Models:**
  - **GCN** → global topology.
  - **GraphSAGE** → inductive learning.
  - **GAT** → neighbor importance.
- **Fusion Layer:**
  - Weighted voting OR transformer-based fusion.
- **Output:** Suspicious node scores + graph-level Trojan probability.

Module 4: Context Expansion

- Select **top-k suspicious nodes** (e.g., k=10).
- Expand **2-hop neighborhood** around each node.
- Package into structured JSON:

```
json
{
  "node_id": 12,
  "type": "XOR",
  "activation_prob": 0.0001,
  "neighbors": [5, 7, 9],
  "distance_to_output": 3
}
```

Module 5: LLM-Driven Reasoning

- **LLM Input:** Suspicious region JSON + RTL snippet.
- **Prompt Template:**

```
Code
```

Analyze the following hardware region:

- Node type: XOR

- Activation probability: 0.0001

- Distance to output: 3

- Neighbors: AND, Register

Question: Is this node part of a Trojan trigger? Explain and suggest a fix.

- **LLM Output:**
    - Verdict (Trojan / benign / uncertain).
    - Explanation (human-readable).
    - Fix suggestion (code-level or design-level).

Module 6: Bias & Consistency Control

- Query LLM twice:
    - Orientation A (infected region first).
    - Orientation B (swapped).
- Add **cross-model consistency** (query two LLMs or two temperature settings).
- Compare verdicts → confidence scoring.

Module 7: Feedback Loop

- Verified LLM explanations → fed back into GNN training as **soft labels**.
- Use **contrastive learning** to align GNN embeddings with LLM semantic embeddings.
- Over time, reduces reliance on LLM inference.

📊 **Workflow Summary**

1. **RTL → Graphs** (structural, functional, temporal).

2. **Feature extraction** → node feature matrix.
3. **Multi-view GNN ensemble** → suspicious node scores.
4. **Context expansion** → JSON + RTL snippets.
5. **LLM reasoning** → explanation + fix.
6. **Bias control** → swap consistency + cross-model check.
7. **Feedback loop** → retrain GNN with LLM insights.

✅ **Tech Stack**

- **Parsing:** PyVerilog, Yosys.
- **Graph ML:** PyTorch Geometric, DGL.
- **LLM:** GPT-4, Claude, or local Hugging Face models.
- **Training:** PyTorch Lightning for modular training.
- **Evaluation:** TrustHub, HT-Pred datasets.